

PROJET ATCE



Asservissement numérique d'un oscillateur à quartz

Nom	N° étudiant
Yasser KHEDDAOUI	22308494
Abdelouaheb BELARBI	22407727
Steeve FEUKAM	22509183
Victoire KOOH	22408624
Fatoumata TRAORE	22307578

Encadrés par : M. J.-M FRIEDT



Table des matières

1	Introduction	1
2	Analyse Préliminaire	2
3	Architecture du circuit	5
3.1	Schéma	5
3.2	Empreintes	6
3.3	PCB	7
3.4	Estimation du coût des composants	9
3.5	Le coût de fabrication	9
3.6	Conception mécanique	10
4	Modélisation SPICE	12
4.1	Configuration du signal DC produit par la PWM	12
4.2	Influence de la varicap sur la résonance du quartz	14
5	Soudure des composants	15
6	Programmation	17
6.1	Bootloader	17
6.2	LED	17
6.3	PWM	18
6.4	Input Capture	19
7	Stabilisation et asservissement de l'oscillateur	21
7.1	Point de fonctionnement :	22
7.2	Réponse indicielle :	23
7.3	Implémentation du PI en boucle fermée :	24
7.4	Test :	25
8	Conclusion	26
	Table des figures	27
9	Références	28

1 | Introduction

La maîtrise du temps et de la fréquence constitue depuis plus d'un siècle un enjeu central pour les télécommunications, la navigation et les systèmes embarqués. Les fondements scientifiques de cette maîtrise remontent aux travaux de Pierre et Jacques Curie, qui mettent en évidence la piézoélectricité en 1880 [1]. Un an plus tard, Lippmann formalise l'effet inverse [3], ouvrant la voie à l'utilisation des cristaux comme transducteurs électromécaniques réversibles. Au début du XX^e siècle, Langevin exploite ces propriétés pour concevoir les premiers dispositifs ultrasonores [2], marquant le passage de la piézoélectricité d'un phénomène de laboratoire à une technologie stratégique. Parmi les matériaux piézoélectriques, le quartz s'impose comme une référence de fréquence [7] et les oscillateurs à quartz [6] deviennent essentiels pour stabiliser la fréquence des émetteurs radio dès les années 1920. Cependant, un oscillateur à quartz est en général précis, mais pas parfaitement exact, car sa fréquence dépend de plusieurs facteurs comme la température, le vieillissement ou les contraintes mécaniques.

C'est dans ce contexte que s'inscrit le présent projet de l'UE Atelier de Conception Électronique du dernier semestre de la licence Électronique et Automatique.

Le sujet proposé consiste à concevoir une carte électronique intégrant un micro-contrôleur ATmega32U4 chargé d'asservir la fréquence du quartz qui le cadence sur un signal stable fourni par un récepteur GPS. Le principe consiste à comparer la fréquence du quartz avec celle du signal de référence GNSS [5], basé sur des horloges atomiques qui sont exactes. Cette comparaison permet de mesurer un écart, ensuite utilisé pour générer un signal de correction appliqué à l'oscillateur afin de compenser ses dérives. L'objectif est de combiner les avantages des deux systèmes en corrigeant le quartz à l'aide de la référence GNSS.

Pour pouvoir atteindre notre objectif, nous avons eu 5 séances théoriques ainsi qu'une semaine de séances pratiques en autonomie. Nous avons réalisé toutes les étapes du projet : l'identification des composants, leur choix, l'architecture du circuit, le routage du PCB, les simulations ainsi que la programmation. Nous détaillerons toutes ces étapes dans ce rapport.

Afin de faciliter la lecture, le rapport suit le déroulement chronologique de notre travail.

2 | Analyse Préliminaire

Pour obtenir une mesure précise de fréquence, on mesure le nombre d'oscillation du quartz entre deux impulsions PPS. L'oscillateur à quartz est composé d'un résonateur mis en parallèle avec un amplificateur. Pour avoir une oscillation dans un oscillateur, il faut que le gain de l'amplificateur puisse compenser les pertes du quartz et que la phase totale de 0 modulo 2π (Condition de Barkhausen). Or dans cet oscillateur, l'amplificateur introduit un déphasage de π et le quartz seul ne permet pas de compenser ce déphasage. Pour respecter la condition de phase imposé par Barkhausen, il est nécessaire d'ajouter des éléments supplémentaires dans le montage. Les condensateurs de pied permettent d'assurer ce rôle.

- **Les condensateurs de pied** : placés d'une part et d'une autre du quartz, associés au quartz permettent de satisfaire la condition de Barkhausen. Le circuit formé par le quartz et les condensateurs de pied apporte le déphasage complémentaire permettant de compenser celui introduit par l'amplificateur et d'obtenir une phase totale de 0 modulo $[2\pi]$. Cet ensemble permet aussi de fixer la capacité de charge du quartz, ce qui assure la stabilité et la précision de la fréquence d'oscillation.

Malgré l'utilisation des condensateurs de pied, la fréquence d'oscillation peut varier en fonction des conditions environnementales. La varicap permet de contrôler cette perturbation de la fréquence.

- **La varicap** : est une diode utilisée comme un condensateur variable commandé en tension. Sa capacité varie en fonction de la tension appliquée à ses bornes en polarisation inverse. Dans l'oscillateur, elle permet de modifier la capacité de charge du quartz de la même manière que les condensateurs de pied, mais de façon ajustable. Elle permet de régler finement la fréquence d'oscillation du quartz afin de corriger les variations de fréquence et d'assurer une synchronisation précise avec le signal PPS du GPS.

La varicap nécessite une tension continue pour fonctionner, cette tension permet de piloter la capacité de la varicap et donc d'ajuster la fréquence d'oscillation. Or, le microcontrôleur est un composant numérique qui génère un signal PWM, c'est-à-dire un signal carré (0 à 5V). Afin d'obtenir une tension continue à partir de ce signal, on utilise un circuit RC.

- **Circuit RC** : est un filtre passe-bas. Il joue ici le rôle de convertisseur numérique–analogique en transformant le signal PWM en une tension continue.

En effet, le filtre RC présente une forte impédance de sortie. S'il est connecté directement à la varicap, cela peut perturber son fonctionnement. Pour éviter cela, on introduit un suiveur.

- **Suiveur** : sert à isoler le circuit RC de la varicap. Grâce à sa forte impédance d'entrée et à sa faible impédance de sortie, il permet de transmettre la tension de commande sans perturber le fonctionnement de la varicap.

Le signal radiofréquence (16 MHz) issu de l'oscillateur peut remonter vers le circuit de commande de la varicap et ainsi perturber le suiveur. Pour éviter cette perturbation, une inductance est ajoutée entre la sortie du suiveur et la varicap. De plus, la tension continue utilisée pour polariser la varicap ne doit pas perturber le fonctionnement de l'oscillateur. Afin d'éviter cela, un condensateur est placé entre la varicap et l'oscillateur.

- L'inductance permet d'empêcher la propagation du signal radiofréquence vers le circuit de commande tout en laissant passer la tension continue nécessaire au pilotage de la varicap.
- Le condensateur permet de découpler les différentes parties du circuit en bloquant la composante continue tout en laissant passer les variations du signal. Il permet donc d'isoler la polarisation de la varicap du reste de l'oscillateur.

L'ensemble de ce montage nécessite une alimentation ainsi qu'un moyen de communication avec l'ordinateur. Ces fonctions sont assurées par le connecteur USB.

- **Le connecteur USB** : fournit une tension de 5V (V_{bus}) utilisée pour alimenter le microcontrôleur, à travers sa broche V_{cc} , ainsi que le reste du circuit. Par ailleurs, il permet également d'établir communication via les lignes des données D+ et D-. Ces lignes sont utilisées pour échanger des informations entre le microcontrôleur et l'ordinateur à l'aide d'une bibliothèque comme LUFA. Des résistances de 22 Ω sont placées en série sur les lignes D+ et D- afin d'améliorer la qualité du signal en limitant les perturbations lors de la transmission des données .

Le microcontrôleur dispose de broches de contrôle qui permettent d'assurer son initialisation et sa mise en programmation. Il s'agit des broches RESET et HWB qui sont actifs à l'état bas.

- **RESET** : Cette broche permet de réinitialiser le microcontrôleur. Un bouton poussoir est relié à la broche RESET afin de permettre une réinitialisation manuelle du microcontrôleur. Lors de l'appui, la broche RESET est mise à l'état bas, ce qui déclenche le redémarrage du système. Nous avons ajouté une résistance de tirage malgré la présence d'une résistance interne dans l'ATmega, par mesure de sécurité supplémentaire.
- **HWB** : Hardware Boot permet de sélectionner le mode de démarrage du microcontrôleur. Lorsqu'elle est activée à l'état bas au moment du reset, elle permet d'activer le bootloader, utilisé pour programmer le microcontrôleur via l'USB. Comme pour le RESET, nous avons utilisé une résistance de tirage afin de garantir un état logique stable en l'absence d'appui sur le bouton.

En plus des broches de contrôle, certaines broches du microcontrôleur sont dédiées à l'amélioration de son fonctionnement interne et à l'interaction avec l'utilisateur.

- Quatre LED connectées aux broches PF1, PF4, PF5 et PF6 du microcontrôleur sont utilisées comme indicateurs visuels. Elles permettent de vérifier rapidement le bon fonctionnement du système sans utiliser d'instrument de mesure
- Par ailleurs, un condensateur est placé sur la broche AREF afin de stabiliser la tension de référence utilisée par le convertisseur analogique-numérique, ce qui permet d'améliorer la précision des mesures.
- Un autre condensateur est connecté à la broche UCAP afin de stabiliser la tension interne nécessaire au fonctionnement du module USB du microcontrôleur, garantissant ainsi une communication fiable.
- Enfin, un connecteur coaxial est relié à la broche PD4. Il est utilisé pour visualiser ou mesurer un signal à l'aide d'un instrument externe comme un oscilloscope.

3 | Architecture du circuit

3.1 | Schéma

Nous avons commencé par identifier où notre circuit résonateur principal devait être connecté sur notre puce ATmega, en parcourant la datasheet [4]. Nous trouvons les broches suivantes que nous allons utiliser :

- **XTAL1/2** : Entrée/sortie de l'amplificateur inverseur que nous connecterons au quartz, et entrée du circuit d'horloge interne.
- **VBUS** : Entrée de surveillance USB.
- **D+/-** : Ports de données USB positif/négatif en amont, à connecter via des résistances de $22\ \Omega$.
- **(A)VCC** : Broche d'alimentation pour le CAN, nous ne l'utilisons pas donc nous la connectons avec le reste du VCC à 5V.
- **AREF** : Référence analogique du convertisseur A/N.
- **UCAP** : Régulateur des pads USB, nous le connectons à un condensateur de $1\mu F$ comme indiqué dans la datasheet.
- **PD7** : Sortie PWM ayant accès au timer 4, dont nous discuterons plus en détail par la suite. Nous le connectons à notre ampli-op après un filtre RC.
- **HWB** : Broche de démarrage matériel, active à l'état bas, nécessaire pour le bootloader que nous aborderons plus tard.
- **RESET** : Broche de réinitialisation du microcontrôleur.
- **(U)GND** : Masse des pads USB et masse générale.

Avec **KiCad**, nous avons commencé par rechercher les composants nécessaires dans le menu des composants, et les avons placés approximativement aux emplacements souhaités. Nous bloquons les entrées que nous n'utiliserons pas, comme la plupart des broches de l'ATmega. Nous veillons également à placer des points de test dans les sections délicates du circuit, comme la sortie de la PWM et après certains filtres, afin de pouvoir visualiser et rectifier tout problème ultérieurement. Nous vérifions l'absence d'erreurs en lançant le vérificateur de règles électriques, puis nous passons à l'assignation des empreintes.

Le schéma suivant présente le circuit électrique du projet :

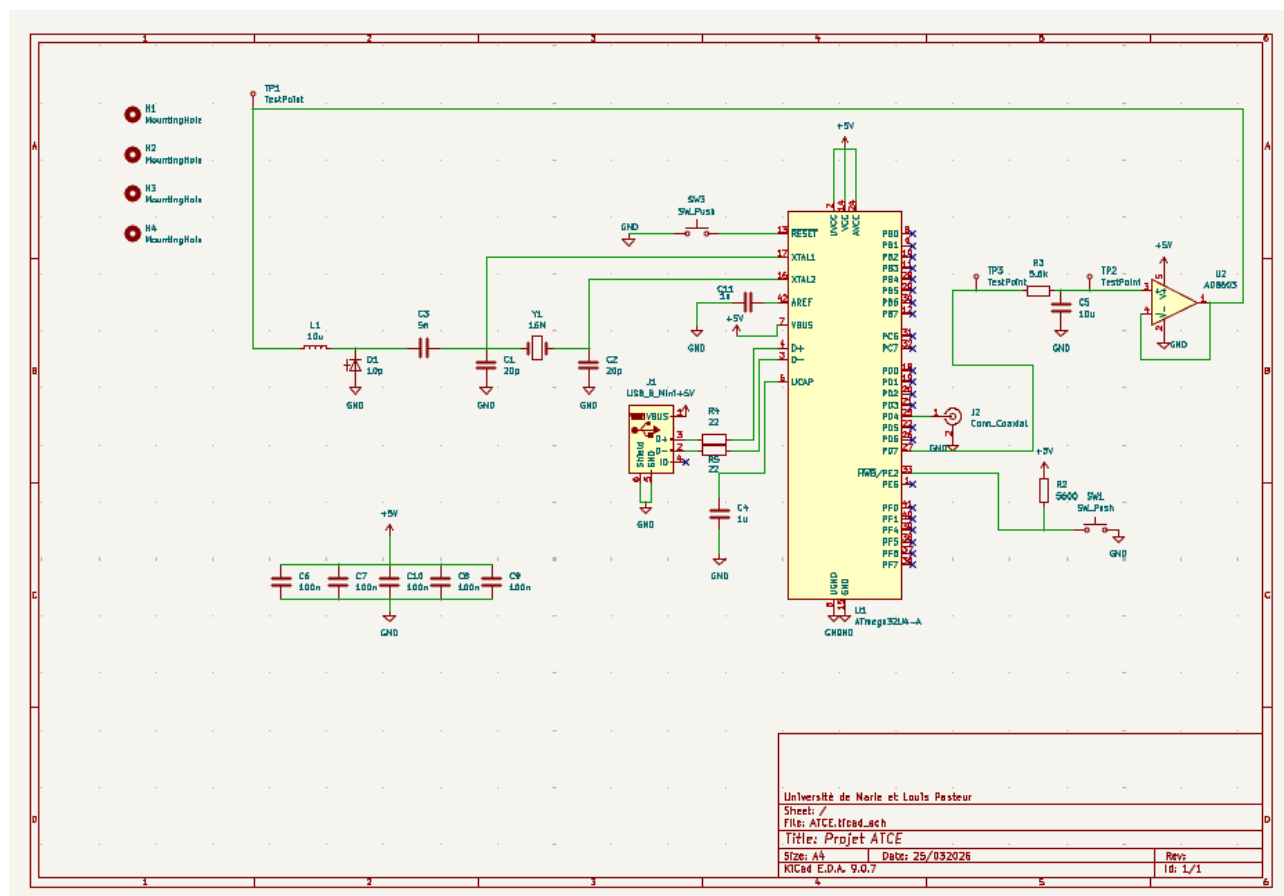


FIGURE 3.1 – Schéma électrique du projet

3.2 | Empreintes

- **Microcontrôleur :** Nous utilisons l'ATmega32U4 tel qu'imposé par le projet, avec lequel nous sommes familiers. Nous trouvons une version en montage en surface de 10mm×10mm qui correspond bien à ce que nous avons prévu.
- **Quartz :** Nous trouvons une version en montage en surface, le HC49-SD. Nous avons sélectionné celui portant le tag *handsoldering* pour faciliter le placement.
- **Varicap :** Nous lui attribuons une empreinte de diode en montage en surface de taille 0805, comme la plupart de nos autres composants passifs.
- **USB :** empreinte correspondant au connecteur USB retenu.
- **Condensateurs, résistances et diodes :** Aucune contrainte particulière autre que le montage en surface, les valeurs correctes et une taille de 0805.

Voici la liste des empreintes finales :

Symbol : Footprint Assignments		
1	C1 -	20p : Capacitor_SMD:C_0805_2012Metric
2	C2 -	20p : Capacitor_SMD:C_0805_2012Metric
3	C3 -	5n : Capacitor_SMD:C_0805_2012Metric
4	C4 -	1u : Capacitor_SMD:C_0805_2012Metric
5	C5 -	10u : Capacitor_SMD:C_0805_2012Metric
6	C6 -	100n : Capacitor_SMD:C_0805_2012Metric
7	C7 -	100n : Capacitor_SMD:C_0805_2012Metric
8	C8 -	100n : Capacitor_SMD:C_0805_2012Metric
9	C9 -	100n : Capacitor_SMD:C_0805_2012Metric
10	C10 -	100n : Capacitor_SMD:C_0805_2012Metric
11	C11 -	1u : Capacitor_SMD:C_0805_2012Metric
12	D1 -	10p : Diode_SMD:D_0805_2012Metric
13	H1 -	MountingHole : MountingHole:MountingHole_2.5mm
14	H2 -	MountingHole : MountingHole:MountingHole_2.5mm
15	H3 -	MountingHole : MountingHole:MountingHole_2.5mm
16	H4 -	MountingHole : MountingHole:MountingHole_2.5mm
17	J1 -	USB_B_Mini : Connector_USB:USB_Mini-B_Lumberg_2486_01_Horizontal
18	J2 -	Conn_Coaxial : Connector_Coaxial:SMA_Amphenol_901-143_Horizontal
19	L1 -	10u : Inductor_SMD:L_0805_2012Metric
20	R2 -	5600 : Resistor_SMD:R_0805_2012Metric
21	R3 -	5.6k : Resistor_SMD:R_0805_2012Metric
22	R4 -	22 : Resistor_SMD:R_0805_2012Metric
23	R5 -	22 : Resistor_SMD:R_0805_2012Metric
24	SW1 -	SW_Push : Button_Switch_SMD:SW_Push_1P1T_NO_CK_RMR2
25	SW3 -	SW_Push : Button_Switch_SMD:SW_Push_1P1T_NO_CK_RMR2
26	TP1 -	TestPoint : TestPoint:TestPoint_Pad_1.0x1.0mm
27	TP2 -	TestPoint : TestPoint:TestPoint_Pad_1.0x1.0mm
28	TP3 -	TestPoint : TestPoint:TestPoint_Pad_1.0x1.0mm
29	U1 -	ATmega32U4-A : Package_QFP:TQFP-44_10x10mm_P0.8mm
30	U2 -	AD8603 : Package_TO_SOT_SMD:TSOT-23-5
31	Y1 -	16M : Crystal:Crystal_SMD_HC49-SD_HandSoldering

FIGURE 3.2 – Liste des empreintes

3.3 | PCB

Après avoir choisi les empreintes représentatives, nous ouvrons un fichier PCB et importons les informations de notre schéma. Nous jouons ensuite sur le placement des composants, en veillant à respecter certains placements primordiaux :

- Le placement le plus critique : positionner le quartz, les capacités de pieds et la varicap aussi près que physiquement possible de l'ATmega, en priorisant le quartz.
- Les condensateurs de découplage doivent être placés directement à côté des broches VCC/AVCC de l'ATmega ; ce sont des réservoirs d'énergie locaux, et une distance trop grande réduit leur efficacité.
- Nous plaçons le connecteur USB symétriquement entre les ports D+ et D-.
- Nous éloignons le connecteur coaxial et l'ampli-op du connecteur USB, car ce sont des parties sensibles de notre circuit, afin de réduire tout effet de bruit provenant de l'USB.

Nous passons ensuite au tracé du contour de notre carte. Nous essayons bien entendu de minimiser la surface tout en restant proche des dimensions standard afin de limiter le coût de fabrication. Nous parvenons à la réduire à 47mm×47mm ; nous estimons qu'elle pourrait être réduite davantage, mais cela rendrait le routage beaucoup plus difficile. Si nous devons réellement faire fabriquer cette carte, nous opterions pour le format standard de 50mm×50mm.

Une étape importante consiste à créer un plan de masse sur notre carte, en déposant une couche de cuivre reliant tous les composants qui en ont besoin. Nous passons ensuite au routage. KiCad est très efficace pour déterminer un chemin optimal entre les composants. Il est à noter que nous avons veillé à ce que les longueurs des pistes D+ et D- vers le connecteur USB soient identiques. Nous obtenons ainsi le résultat suivant :

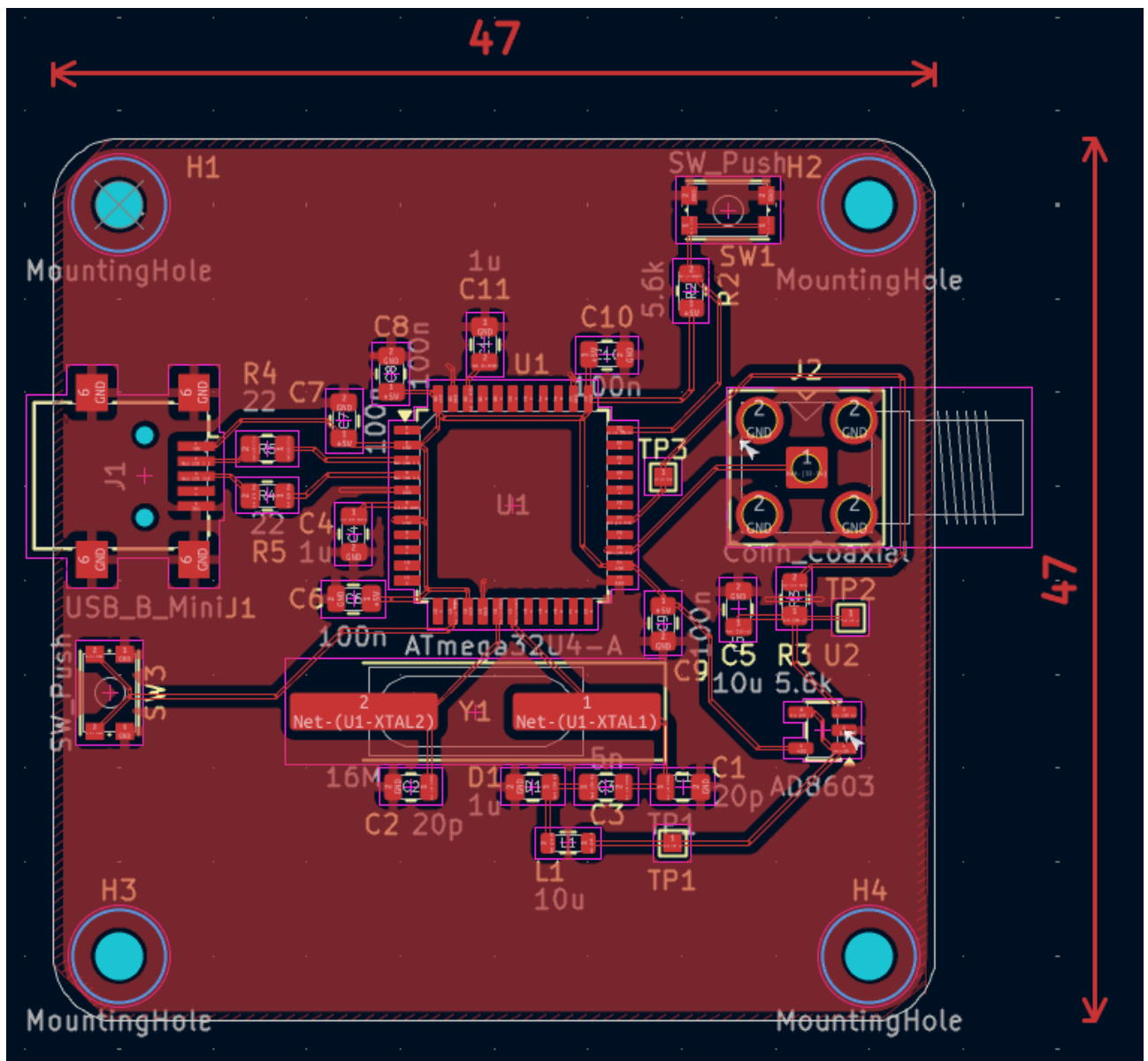


FIGURE 3.3 – PCB du schéma électrique

3.4 | Estimation du coût des composants

Reference	Qty	Value	Cout	FARNELL	Code commande	Footprint
C1,C2	2	12p	(+1) 0.240 €	KAM31BCG2H120JT	4758913	C_1206
C3	1	5n	(+1) 0.21 €	08055C502KAT2A	3359766	C_0805
C4,C10	2	1u	(+1) 0.332 €	AS0805KKX7R8BB105	4048356	C_0805
C5-C11	6	100n	(+1) 0.058 €	MC0805B104K500CT	2310707	C_0805
D1	1	1u	(+5) 1.48 €	BB535E7904HTSA1	2443452	SOD-323
D2-D6	4	LED	(+5) 0.255 €	KP-1608SURC	8529868	LED_0805
J1	1	SMA	(+1) 3.78 €	CONSMA002SMDGT	4218722	SMA
J3	1	USB Mini	(+1) 1.42 €	65100516121	1642036	USB_Mini-B
L1	1	10uH	(+1) 0.14€	LSQNA251818T100K	4048028	0805
R1,R2	2	5.6k	(+1) 0.0078 €	RC0805JR-075K6L	4013417	R_0805
R3,R4	2	22	(+1) 0.0277 €	CR0805-FX-22R0ELF	4153658	R_0805
SW1,SW2	2	SPST	(+10) 0.288 €	SKHHALA010	2056809	Header
TP1-TP4	2	TestPoint	(+1) 0.355 €	5016	2292858	TP
U1	1	ATmega32U4	(+1) 4.76 €	ATMEGA32U4-AUR	2443182	TQFP-44
U2	1	AOP	(+1) 1.15 €	MCPCV96UT-E/OT	3758609	SO-8
Y1	1	16MHz	(+1) 0.27 €	ABLS-16.000MHZ-B2-T	2467720	Crystal

TABLE 3.1 – Les prix indiqués sont issus du fournisseur **Farnell**.

Le coût des composants a été calculé à partir de la liste finale en tenant compte des quantités nécessaires ainsi que des contraintes de commande du fournisseur (achat par lots pour certains composants). Le coût total des composants s'élève à environ 16,03 € et en ajoutant les frais de livraison estimés à 11.99 €, le coût global est estimé à 28,02 €.

3.5 | Le coût de fabrication

Pour budgéter le coût de fabrication de notre circuit, nous avons utilisé le calculateur du site **Eurocircuits**. Cet outil permet d'estimer le prix de fabrication d'un circuit imprimé en téléchargeant directement le fichier au format `kicad_pcb`. Les estimations ont été réalisées pour notre PCB à 2 couches de dimensions 47 mm × 47 mm et conçu en classe 6. Il est important de noter que le coût dépend également des dimensions du circuit. En effet, un PCB plus petit, comme celui proposé par notre enseignant (40.5 mm × 35 mm), est moins coûteux à fabriquer.

D'après les résultats obtenus (voir Tableau 3.2), le coût dépend principalement du délai de fabrication et du nombre de circuits commandés. On note que pour un seul PCB, le prix diminue fortement entre un délai de 2 jours et 3 jours, puis reste constant à partir de ce délai.

TABLE 3.2 – Estimations du coût de fabrication (prix TTC)

X	1 PCB	10 PCB	100 PCB
2 jours	126.38€	249.14€	753.83€
3 jours	51.44€	198.32€	602.58€
4 jours	51.44€	147.38€	452.54€
5 jours	51.44€	104.67€	310.97€
7 jours	51.44€	104.67€	310.97€

Les estimations montrent que pour un seul PCB, le coût minimal est de 51.44€ TTC (soit 42.51€ hors taxes) pour un délai de fabrication de 3 jours et au-delà de ce délai le prix ne diminue plus pour une seule carte. Pour des quantités plus importantes, on remarque que le prix continue de diminuer jusqu'à une expédition sous 5 jours. Par exemple, pour 100 PCB, passer d'un délai de 2 jours à 5 jours permet d'économiser environ 442.86€. De même, pour 10 PCB, l'économie est d'environ 144.47€. Donc choisir un délai de fabrication légèrement plus long permet de réduire le coût de fabrication, en particulier pour des commandes en grande quantité. Ainsi, la réduction des dimensions du PCB permet également de diminuer les prix.

3.6 | Conception mécanique

Afin de valider l'intégration mécanique de la carte, nous avons réalisé la modélisation du boîtier dans **FreeCAD**. Il s'agit d'un logiciel libre et open source de conception 3D paramétrique. Il permet de créer des modèles solides à partir d'esquisses contraintes, d'effectuer des opérations comme l'extrusion, l'enlèvement de matière ou encore la création de congés ou chanfreins.

L'objectif de cette étape était de concevoir un boîtier fonctionnel, capable d'accueillir la carte électronique réalisée sous KiCad.

Pour assurer la cohérence entre la conception électronique et la modélisation mécanique, nous avons installé l'atelier KiCadStepUP dans FreeCAD ce qui nous a permis d'importer non seulement le contour de PCB mais également la position des composants, les empreintes 3D associées et l'orientation des connecteurs.

La démarche a consisté à :

Générer le volume du boîtier à partir de l'atelier Part et par opérations booléennes (soustraction et union), vérifier la cohérence dimensionnelle du PCB puis ajouter les ouvertures nécessaires (USB, boutons, câble coaxial) et intégrer des éléments de fixation (piliers, perçage, logements pour les vis)

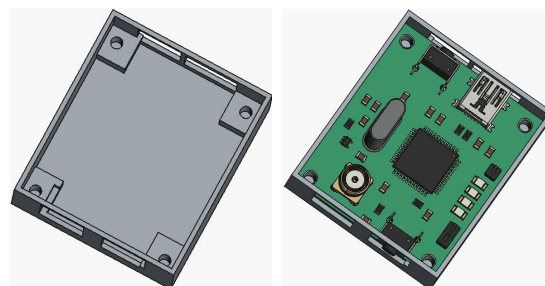


FIGURE 3.4 – Vues du boîtier modélisé sous FreeCAD

Afin de protéger les composants, nous avons aussi réaliser un couvercle en utilisant les mêmes démarches (atelier Part, opérations booléennes (union, soustraction)).

Nous avons adapté le boîtier et son couvercle à notre carte, ce qui explique l'ouverture du boîtier et du couvercle au niveau du câble coaxiale car celui que nous avons utilisé lors de la réalisation de la carte est horizontal.

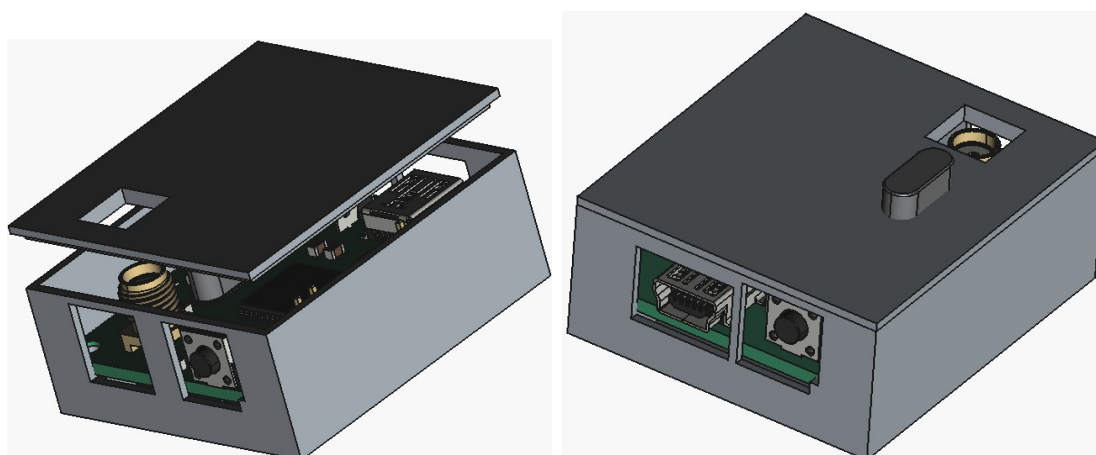


FIGURE 3.5 – Modélisation 3D du boîtier

À noter que la taille du résonateur que nous avons utilisé lors de la soudure de notre carte n'est pas celle qui est actuellement représentée sur l'image (à droite), en effet le notre est bien plus bas et pourra donc tenir sous le couvercle.

4 | Modélisation SPICE

Dans cette partie du projet, après avoir tracé le circuit principal sur KiCad ainsi que son PCB, et avant de finaliser toutes nos valeurs et de passer à la soudure, nous avons dû vérifier certains paramètres délicats de notre circuit. Principalement la conformité de notre sortie de modulation de largeur d'impulsion, qui nous permet de confirmer les valeurs du filtre RC, ainsi que la variance maximale que notre varicap peut produire sur la fréquence de résonance de 16 MHz à laquelle fonctionne notre quartz. Pour effectuer ces manipulations, nous avons utilisé les fonctions SPICE intégrées dans KiCad, afin de visualiser les allures attendues et de vérifier notre travail. Nous avons choisi de dessiner deux mini-circuits avec certaines équivalences qui nous permettent d'approcher le comportement réel autant que possible.

4.1 | Configuration du signal DC produit par la PWM

Nous commençons par dessiner le circuit équivalent de la sortie de notre broche PD7, jusqu'à la varicap (voir figure 4.1), où nous nous attendons à obtenir un signal DC avec une fluctuation ΔV aussi faible que possible.

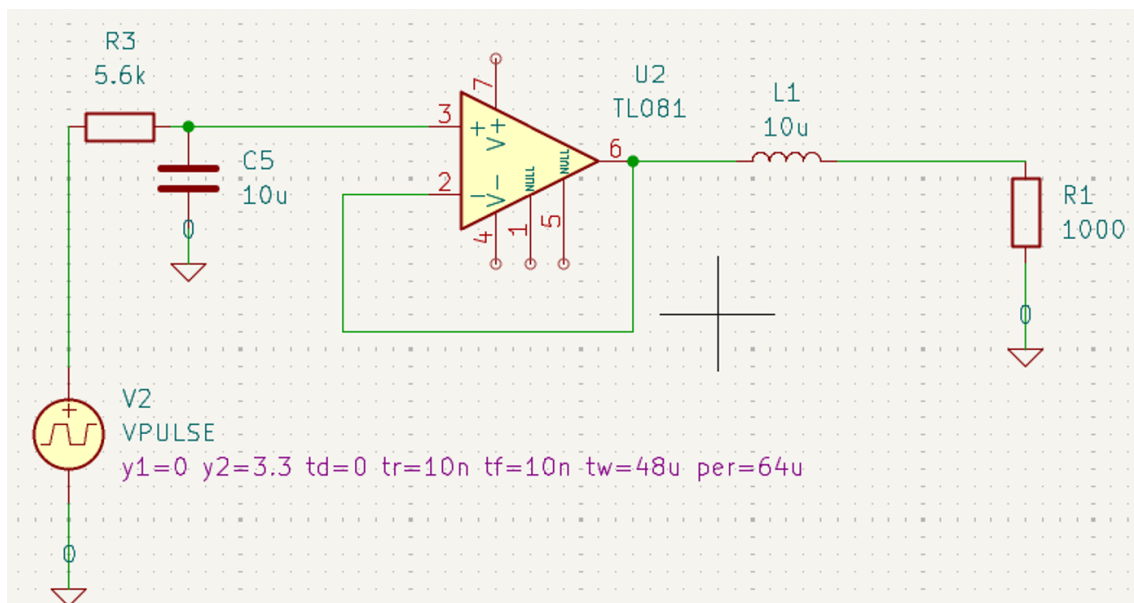


FIGURE 4.1 – Circuit équivalent de la sortie PB5

Nous avons remplacé la sortie PWM par un signal V_{pulse} , et avons inséré une netlist qui simplifie le comportement de notre ampli-op en ne conservant que 2 entrées : inv(2), non-inv(3) et la sortie(6). Nous remplaçons également toutes les masses par le GND spécifique à SPICE.

Pour déterminer la fréquence de V_{pulse} , nous avons calculé f_{pwm} en tenant compte des valeurs de OCR4C et OCR4D ; qui sont deux registres contrôlant à la fois la fréquence

de notre PWM et le rapport cyclique, selon la relation $\alpha = \frac{OCR4D}{OCR4C}$, ce qui implique que $OCR4C$ est toujours supérieur à $OCR4D$.

Il est à noter que le timer utilisé dans l'ATmega32U4 est un timer haute vitesse unique, disposant de 2 bits supplémentaires par rapport aux 8 bits sur lesquels travaille le micro-contrôleur. Cela permet une plus grande précision de la PWM. Pour exploiter le plein potentiel du timer, on utilise un registre 8 bits supplémentaire TC4H. Seuls les deux bits les plus significatifs de ce registre sont nécessaires pour renseigner les bits de poids fort de notre timer 10 bits, les 8 bits restants étant directement assignés depuis OCR4C.

Nous choisissons de calculer f_{pwm} en mode rapide, c'est-à-dire en utilisant l'équation ci-dessous, avec une valeur de prédiviseur $N=1$ pour conserver la fréquence maximale, et la valeur maximale de $OCR4C$ qui est $2^{10}-1=1023$, afin d'obtenir une précision maximale.

$$f_{pwm} = \frac{f_{quartz}}{N \cdot (OCR4C + 1)} = \frac{16 \times 10^6}{1 \times 1024} = 15625 \text{ Hz}$$

La f_{pwm} ainsi obtenue est infiniment plus grande que le signal 1 PPS qui se rafraîchit à 1 Hz, de sorte qu'en pratique le choix du mode rapide n'a que peu d'impact sur le résultat final.

f_{pwm} étant maintenant connue, nous modifions V_{pulse} en conséquence et lançons une analyse transitoire, en fixant le pas de temps à la période $T=64 \mu s$ et le temps final à une seconde. Nous faisons ensuite varier R et C jusqu'à obtenir des valeurs de $T_{r5\%}$ ΔV satisfaisantes. La figure 4.2 représente le résultat de la simulation :

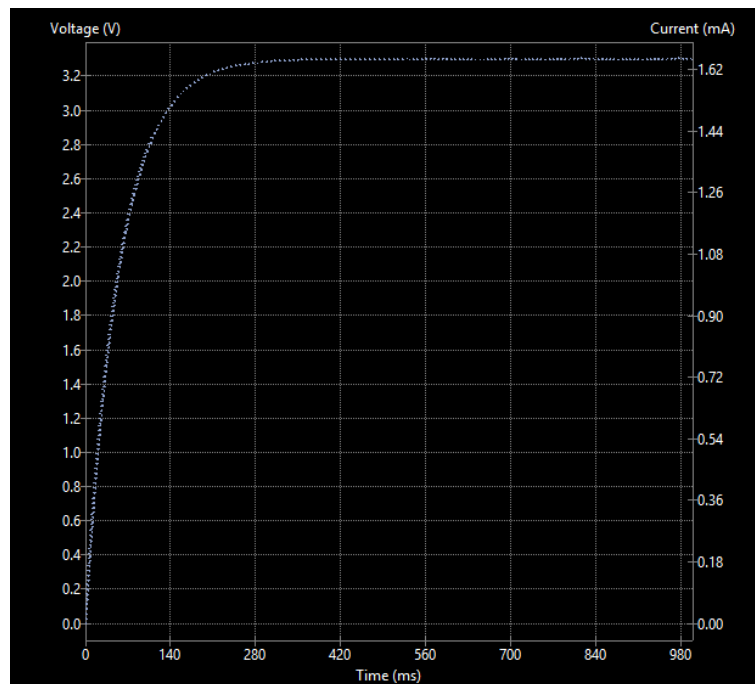


FIGURE 4.2 – Résultat de simulation

Nous obtenons finalement $R = 5.6k\Omega$ et $C=10\mu F$.

4.2 | Influence de la varicap sur la résonance du quartz

Le second circuit de simulation que nous avons réalisé constitue la suite directe du précédent. Nous nous intéressons cette fois à la résonance de notre quartz et à l'amplitude du déplacement que la varicap peut y induire. Cette information est indispensable pour évaluer la robustesse de notre circuit face aux perturbations extérieures, notamment la température, qui peut modifier le comportement du quartz.

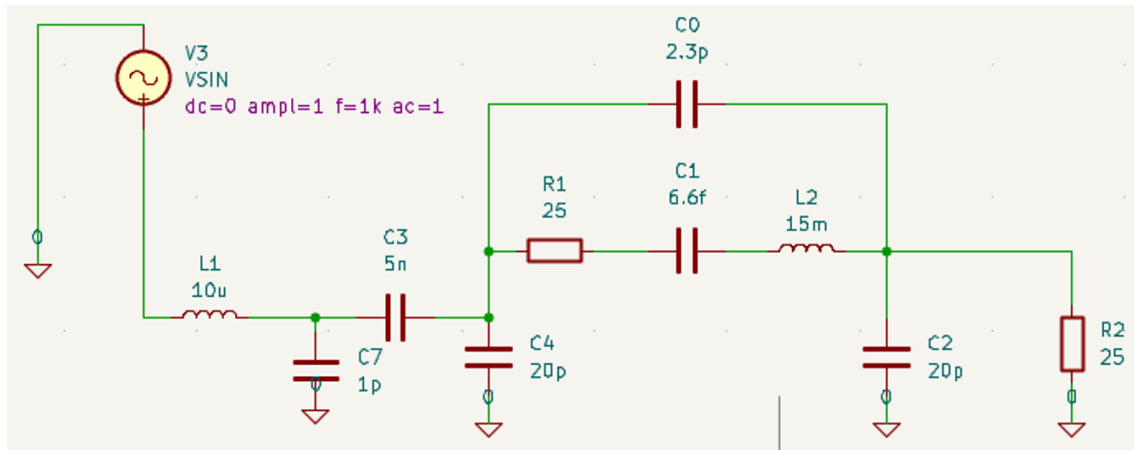


FIGURE 4.3 – Schéma spice du résonnateur

Nous avons remplacé le quartz par un circuit RLC équivalent avec C_1 en série et C_0 en parallèle. Nous déterminons les valeurs de R et C_0 à partir de la datasheet associée au quartz, également mentionnée dans le texte du cours. Nous obtenons les valeurs suivantes : $R=25\Omega$, $C_0 = 2.3pF$ et $C_1 = 6.6fF$.

Nous effectuons une analyse AC sur 10000 points entre 15,99MHz et 16MHz afin d'observer le décalage de fréquence. Les valeurs de la varicap doivent être du même ordre de grandeur que les capacités de pieds, nous la faisons donc varier de 1pF à 20pF.

En zoomant et en plaçant des curseurs, nous observons un écart de pic de 3.68kHz (voir figure 4.4), que nous estimons être une plage saine, nous offrant une manoeuvrabilité suffisante pour les étapes suivantes.

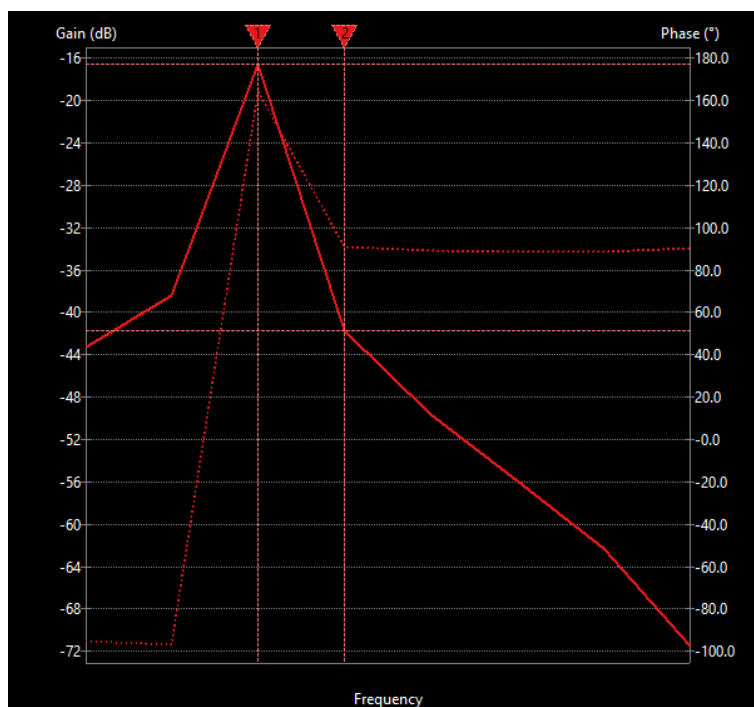


FIGURE 4.4 – Résultat de simulation

5 | Soudure des composants

Pour la soudure des composants, nous avons suivi un ordre logique permettant de faciliter le montage et de pouvoir vérifier le fonctionnement étape par étape. Nous avons commencé par souder le connecteur USB, car il subit des contraintes mécaniques importantes (branchement et débranchement). Le souder en premier permet de bien l'aligner avec le PCB et de le corriger plus facilement si nécessaire. Ensuite, nous avons soudé le microcontrôleur. Nous avons ensuite soudé les condensateurs de découplage, les condensateurs de pied ainsi que les résistances reliant l'USB au microcontrôleur. Cette étape permettait d'alimenter le microcontrôleur et de vérifier la présence de la tension. À l'aide d'un multimètre, nous avons mesuré une tension de 5,14 V au niveau du microcontrôleur (voir figure 5.1). Après cette vérification, nous avons poursuivi la soudure des autres composants : les LED, la varicap en respectant sa polarisation, puis l'amplificateur. Pour la soudure de l'amplificateur, nous nous sommes appuyés sur le PCB afin de respecter la bonne orientation du composant. A la fin de la soudure, une vérification au microscope a été réalisée afin de s'assurer que toutes les pattes étaient correctement soudées et qu'il n'y avait pas de ponts de soudure.

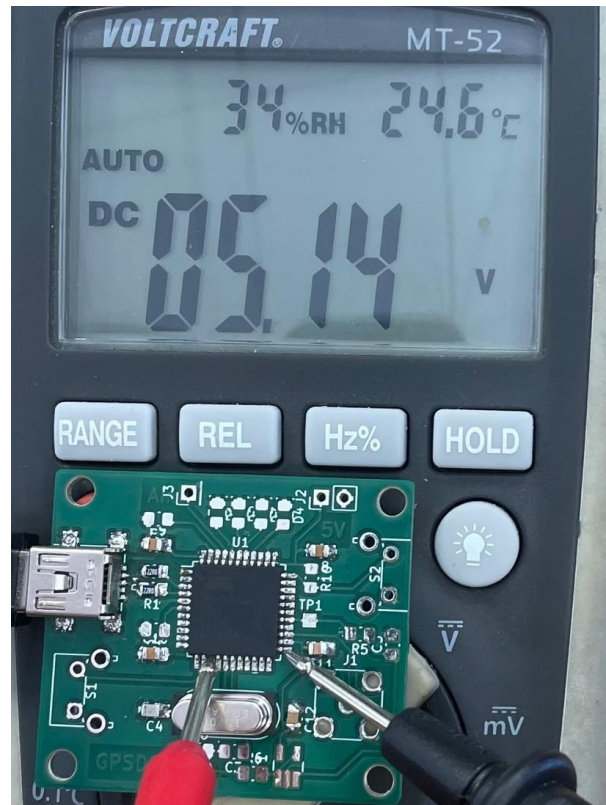


FIGURE 5.1 – Mesure de la tension au niveau du microcontrôleur

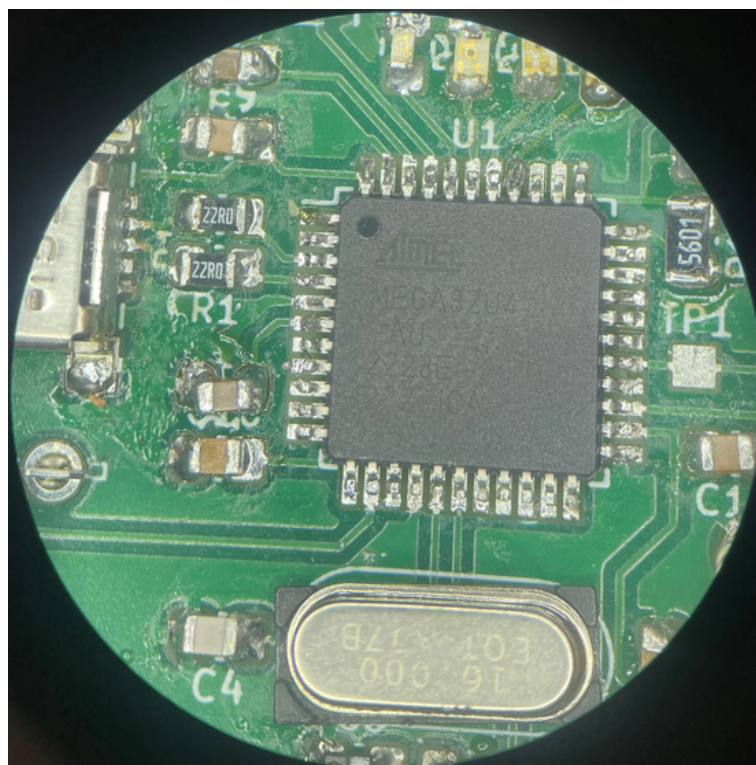


FIGURE 5.2 – Vérification de la carte au microscope

6 | Programmation

6.1 | Bootloader

Le microcontrôleur Atmega32u4 possède par défaut le protocole de communication de chez ATMEL qui est le DFU (Device Firmware Upgrade). Ce bootloader nous permettra de communiquer avec le port USB afin de pouvoir flasher le microcontrôleur au moment de sa mise sous tension. Pour ce faire on utilisera plus précisément l'outil dfu-programmer qui est disponible [en ligne](#). Ce bootloader a la particularité de rechercher dans sa mémoire lors de son démarrage le dernier programme chargé depuis le port USB. Dans un premier temps pour pouvoir flasher la mémoire de notre microcontrôleur il faudra effacer la mémoire flash avant de pouvoir écrire dessus, c'est-à-dire de passer tous les bits à 1 avec la commande Erase puis de flasher l'exécutable avec la commande flash et pour finir on réinitialise le microcontrôleur pour lancer son exécution avec la commande reset.

```
install: $(EXEC).hex
        $(DFU) $(MCU) erase
        $(DFU) $(MCU) flash
        $(EXEC).hex $(DFU) $(MCU) reset
```

Avant de compiler cet ensemble de commandes sur la carte, il faut effectuer une certaine combinaison de touches qui consiste à mettre le $\overline{\text{HWB}}$ à l'état bas donc à l'enfoncer puis d'enfoncer le reset pendant un instant avant de relâcher après et pour finir de relâcher $\overline{\text{HWB}}$. La carte est maintenant prête à recevoir l'exécutable.

6.2 | LED

Dans un premier temps pour pouvoir communiquer avec notre microcontrôleur, nous allons faire clignoter les LEDs situées sur les ports PF[1,2,3,4]. Le microcontrôleur assigne par défaut sur ses broches une interface JTAG qui est en effet une interface matérielle permettant de communiquer avec l'utilisateur s'il le souhaite. Pour configurer nos broches PF en entrée/sortie, on doit pouvoir désactiver ce JTAG. Pour cela on doit confirmer deux fois de suite la désactivation du JTAG en modifiant dans le registre MCUCR le passage à 1 du bit JTD.

```
MCUCR|=(1<<JTD);
MCUCR|=(1<<JTD);
```

Après cela on pourra essayer de lancer notre premier programme sur la carte pour savoir si on est capable de charger un programme sur cette nouvelle carte, le tout en se rappelant des étapes de flash du microcontrôleur. Le code est nommé led.c et est disponible sur [GitHub](#) avec son makefile.

6.3 | PWM

Étant donné que notre microcontrôleur ne possède pas de convertisseur analogique-numérique, la PWM (Pulse Width Modulation) va nous aider à générer un signal variable en sortie de la PWM qui sera ensuite lissé par un filtre passe-bas, on obtiendra une tension continue qui permettra de commander la varicap en faisant varier les conditions de Barkhausen qui nous fera varier le condensateur de pied pour changer la fréquence de l'oscillateur. On retient que ce signal de commande va asservir la fréquence du quartz, il nous faudrait avoir un signal avec une fréquence élevée qui nous permettra de réduire le temps de réaction.

L'objectif est d'obtenir un signal en sortie de la PWM de fréquence élevée, on réglera la PWM en mode Fast. Ce mode exécute une période complète deux fois plus vite que le mode Phase Correct PWM. L'entrée du filtre passe-bas est connectée sur la broche D7 du microcontrôleur et le timer assigné à cette broche est le Timer 4 qui a la particularité d'être sur 10 bits, il possède une résolution de 1024. L'obtention du signal PWM est due à une modulation de largeur d'impulsion, dans notre cas le Timer 4 possède deux registres essentiels le registre OCR4C qui définit la résolution du Timer, c'est-à-dire jusqu'où il peut compter au maximum et le registre OCR4D qui permet de faire varier le rapport cyclique. Le fonctionnement de la PWM est simple, le compteur du Timer 4 compte de zéro jusqu'à la limite du registre OCR4C (1023). Au départ lorsque le compteur s'incrémente le signal commence à former un front montant puis lorsque la valeur du compteur croise la valeur du registre OCR4D, le signal de sortie reste à l'état haut. Le compteur atteint le maximum puis redémarre à zéro.

L'activation de la PWM passe par quelques étapes, tout d'abord il faut savoir quel diviseur de fréquence on veut utiliser, quel mode de la PWM on souhaite utiliser, et pour finir quelle résolution et quel rapport cyclique l'on veut avoir. En ce qui concerne le diviseur de fréquence dans notre cas, comme l'on veut avoir une fréquence en sortie élevée il vaut mieux utiliser un diviseur de fréquence de 1 pour avoir toute la gamme de fréquences disponible. Pour l'activation du mode Fast de la PWM, il faut s'assurer que certains registres comme TCCR4D et TCCR4C soient définis :

```
// PWM4X = 1 : activation du mode fast PWM  
TCCR4D = (1 << PWM4X);
```

```
// OC4D mis a 0 quand OCR4D = TC4 et activation de la PWM  
TCCR4C = (1 << COM4D1) | (1 << PWM4D);
```

La suite de ce code est disponible sur [GitHub](#).

Pour la visualisation de notre PWM, on se munit d'une sonde et d'un oscilloscope et on obtient comme graphe :

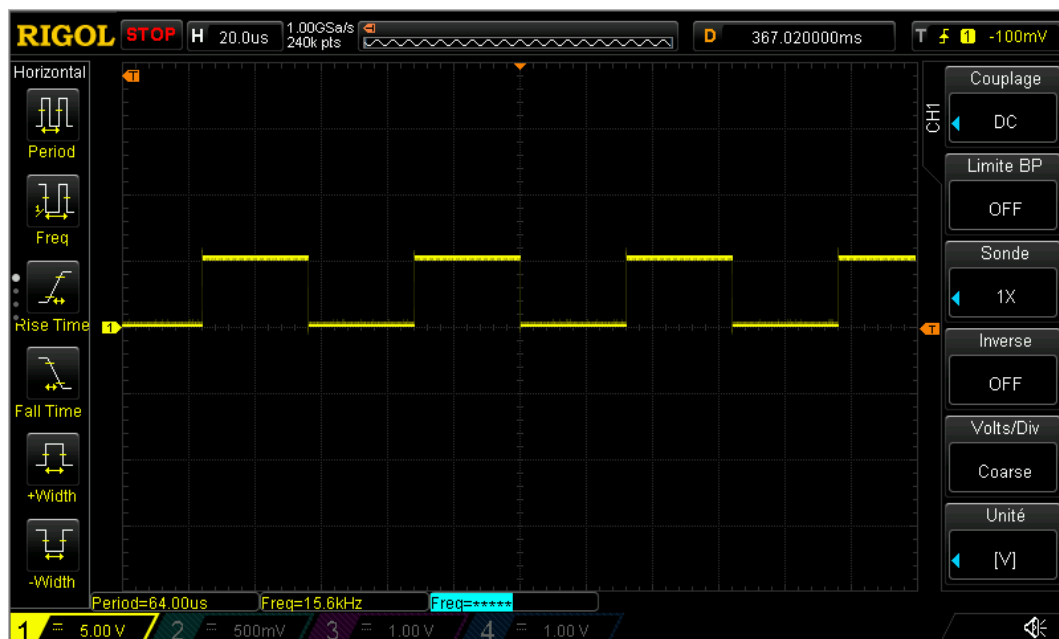


FIGURE 6.1 – Résultat de la Fast PWM

Notre signal de sortie de la PWM possède une amplitude de 5 V et une fréquence de 15,6 kHz. Comme le démontre le calcul de la fréquence de la PWM en mode Fast dans la partie **6.1 - Configuration du signal DC produit par la PWM**. Ce signal de 15,6 kHz aura une vitesse de correction considérable lors de la dérive du signal 1PPS du GPS.

6.4 | Input Capture

Notre signal de PWM étant implémenté, il ne nous reste plus qu'à savoir si on est capable de capturer l'instant où le signal 1PPS du GPS est détecté sur les fronts montants de notre Timer. Comme le Timer 4 est codé sur 10 bits, il ne garantit pas une résolution optimale pour la détection du 1PPS, c'est pour cela qu'on utilisera à la place un timer 16 bits qui a une résolution de 65536. Le Timer 1 fera l'affaire vu qu'il possède une fonctionnalité d'input capture edge select, cette fonctionnalité nous permettra de mesurer l'intervalle de temps entre 2 fronts montants du 1PPS, et comme le Timer est cadencé par la fréquence du Quartz, on représentera directement la fréquence du quartz.

Pour mettre en œuvre notre asservissement en boucle ouverte, c'est-à-dire qu'on va laisser la fréquence dériver librement avec son environnement (température). On va mesurer le nombre d'overflow du timer entre 2 captures du front montant (ce nombre d'overflow n'est autre que le nombre de fois qu'on compte jusqu'à 2^{16}) et ensuite lorsqu'une interruption de capture du front montant du signal 1PPS sera déclenchée, on stockera la valeur du compteur à cet instant et on prendra soin de remettre le compteur du timer et le nombre

d'overflow à zéro pour préparer la prochaine mesure. Le calcul de la fréquence mesurée s'obtient par le calcul suivant :

■ **nOVF** : nombre de débordements du timer (16 bits)

■ **PPS** : valeur du registre de capture (ICR1)

$$f_{\text{mesuree}} = 2^{16} \times \text{nbr OVf} + \text{PPS}$$

Pour l'activation de l'interruption de l'overflow et de l'input capture on se rappellera qu'il faut placer ces registres en sortie :

```
// selection du front montant pour la capture
TCCR1B |= (1 << ICES1);
// - interruption capture (ICIE1)
// - interruption overflow (TOIE1)
TIMSK1 |= (1 << ICIE1) | (1 << TOIE1);
```

Nous avons déposer le code complet sur [Github](#) également.

Pour le test de notre programme il nous faudra simuler le signal 1PPS du GPS et pour cela on se munira d'un générateur. Nous avons régler la fréquence à 1 Hz, l'amplitude à 3,3 V (peak to peak) et un offset de 1.65 V avec un rapport cyclique de 20%.

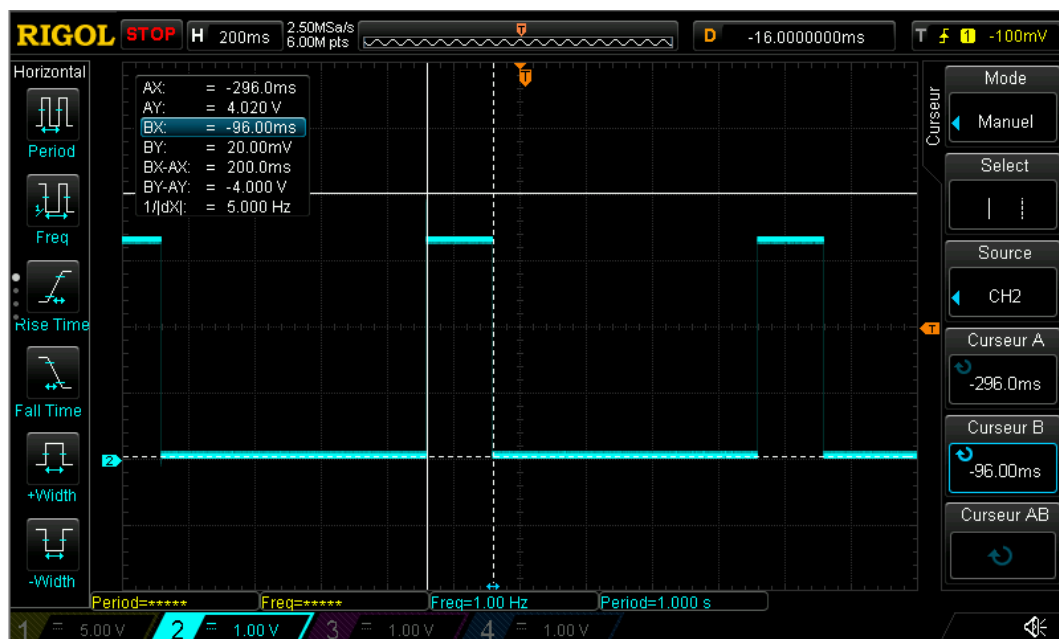


FIGURE 6.2 – Signal 1PPS

Lors de la compilation du programme, on affiche les valeurs mesurées via la communication avec le port série USB et l'interface minicom. Grâce à la commande "minicom -D /dev/ttyACM0" puis "CTRL+A avec L", on enregistre les mesures pour les tracer par la suite avec des logiciels comme GNU PLOT ou MATLAB. Au cours de l'enregistrement des mesures, on remarque que le nombre d'overflow reste constant à une valeur de 244 et

la valeur du compteur ICR1 est de 8759, ce qui nous fait une fréquence de 15999543 Hz. Lorsqu'on pose la pointe de notre fer à souder sur le quartz de notre carte, on observe une baisse de la fréquence puis une augmentation progressive de la fréquence avant une stabilisation à sa valeur nominale lorsqu'on retire la pointe.

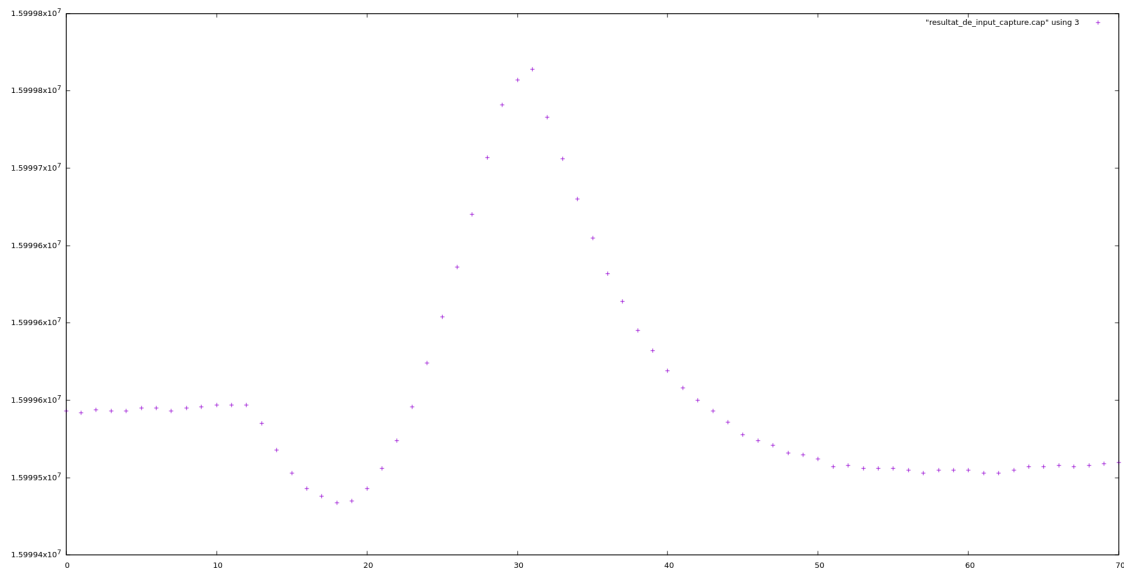


FIGURE 6.3 – Résultat de la mesure d'inputCapture

Lorsqu'on chauffe le quartz, on observe la fréquence du quartz s'écarter d'au moins 2 kHz. Avec cette mesure en boucle ouverte, on est maintenant capable de mesurer la fréquence du quartz et on peut maintenant étudier le système en boucle fermée pour l'asservir et corriger cet écart de fréquence.

7 | Stabilisation et asservissement de l'oscillateur

Maintenant que nous avons une réponse à chaque 1PPS qui varie avec la température, ainsi qu'un moyen de décaler la polarité de notre varicap via le signal PWM, nous aimerions éliminer ces variations aussi efficacement que possible. C'est pourquoi nous allons réguler en utilisant des méthodes classiques de contrôle et commande. Nous avons choisi d'utiliser la méthode suggérée dans le cours, les équations de Takahashi. Pour cela, nous aurions besoin de certaines valeurs importantes spécifiques à notre système, qui sont la pente pour une réponse indicielle α , le retard L et la période d'échantillonnage T dont nous savons déjà qu'elle est égale à 1s puisque nous envoyons une impulsion par seconde. Nous étions également intéressés par la connaissance du point de fonctionnement de notre résonateur, afin de pouvoir le définir comme consigne. Nous procédons aux tests en boucle ouverte suivants.

7.1 | Point de fonctionnement :

Nous avons d'abord commencé par déterminer notre consigne. Pour ce faire, l'idée était de faire varier le rapport cyclique, ce qui signifie faire varier la polarité de la varicap à travers un balayage de toute la plage de ses valeurs ; nous avons décidé d'utiliser un pas de 10, car cela représenterait environ 1%. Pour cela, nous avons utilisé le code d'input capture dont nous avons parlé dans la section précédente, et ajouté quelques éléments : les fonctions PWM, incluant une nouvelle fonction appelée "PWM-rapport-cyclique" qui, comme son nom l'indique, nous permet de varier le rapport cyclique dans la boucle while. Nous affichons la valeur de la fréquence actuelle ainsi que le rapport cyclique, nous incrémentons le rapport cyclique, puis nous l'appliquons à notre nouvelle fonction. Nous obtenons le graphique suivant :

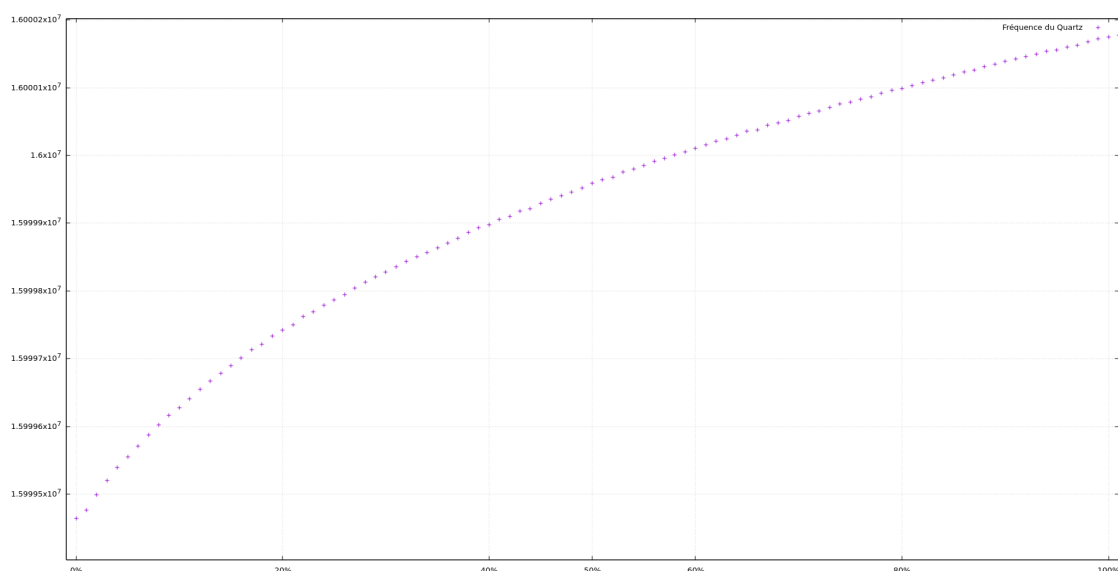


FIGURE 7.1 – Graphique de l'effet sur la fréquence d'un sweep du rapport cyclique, pour trouver le point de fonctionnement 15 999 958 Hz

Comme nous pouvons l'observer, la courbe n'est pas linéaire, mais plutôt parabolique, nous prenons donc la valeur à 50% comme point de fonctionnement, que nous trouvons être la valeur 15 999 958 Hz.

7.2 | Réponse indicielle :

C'est ici que nous trouvons le retard L et la pente α , nécessaires pour les calculs de Takahashi. La façon dont nous avons procédé est la suivante : nous avons visualisé la réponse de notre système à un changement soudain du rapport cyclique qui n'est pas incrémentiel, et extrait les informations du graphique. Nous avons choisi de sauter de 45% (461/1023) à 55% (563/1023), en restant dans chaque état pendant 5s, en utilisant une instruction if et un flag d'état. Pour obtenir un résultat plus précis, nous avons pris 29 mesures consécutives, puis nous en avons fait la moyenne pour trouver le graphique suivant :

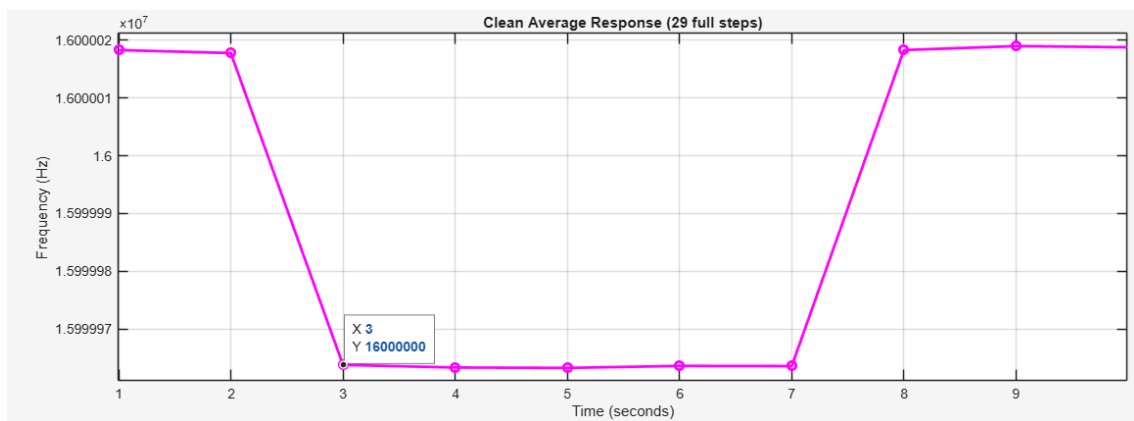


FIGURE 7.2 – Moyenne des 29 mesures effectuées de la réponse indicielle

Nous calculons la pente et trouvons $\alpha = 46$, puis nous zoomons autant que possible pour voir un retard, mais nous voyons que la réponse commence au moment exact où nous envoyons la commande ; nous décidons de prendre $L = 0s$ dans ce cas.

Note : Cette observation que $L = 0s$ et le fait qu'il n'y ait pas eu de dépassement est ce qui nous a conduits à choisir une régulation PI au lieu de passer à un PID, car il semble ne pas être d'une grande utilité ici.

Nous décidons d'appliquer les équations de Takahashi manuellement, en trouvant :

$$k_i = \frac{0.27}{\alpha(L + 0.5 \cdot T_e)^2} = \frac{0.27}{46 \cdot (0.5)^2} = 0.023 \quad (7.1)$$

$$k_p = \frac{0.9}{\alpha \cdot 0.5} - 0.5 \cdot k_i = 0.012 \quad (7.2)$$

Avec ces variables trouvées, nous pouvons procéder à la régulation PI.

7.3 | Implémentation du PI en boucle fermée :

Il est utile de comprendre les effets réels des parties proportionnelle et intégrale de notre régulation. Le terme P réagira presque instantanément à l'erreur causée par nos forces externes, étant capable d'absorber la large variation et de maintenir notre fréquence à la valeur où il l'a trouvée initialement, ce qui serait inférieur à la consigne puisque la fréquence chute d'abord en valeur avant de diverger vers le haut ; c'est là qu'intervient le terme intégral, accumulant l'erreur pendant toute la durée de la perturbation afin d'éliminer l'erreur statique, nous ramenant à la valeur de consigne.

Note : le terme intégral peut également diverger selon la perturbation, nous nous assurons donc de limiter son accumulation par un anti-windup, bloquant sa divergence.

Nous utilisons l'équation présente dans la section 9.6.2 du cours, en excluant le terme dérivé. Nous finissons par utiliser l'équation suivante pour créer la commande PWM :

commande-pwm = kp * (erreur - erreur-old) + ki * erreur ;

Où l'erreur est la différence entre la fréquence mesurée et la consigne, et erreur-old est la valeur de l'erreur de l'impulsion précédente.

L'implémentation a commencé par la définition de nos 3 constantes, ki, kp et la consigne. Nous notons que nous avons multiplié nos coefficients PI par 1000 pour les utiliser comme entiers, car l'atmega aurait des difficultés s'il s'agissait des petites fractions que nous avons trouvées initialement (**Pour la suite, voici le code sur Github**).

Dans la boucle while, nous commençons une mesure avec le pulse flag, comme d'habitude, puis nous définissons les valeurs de l'erreur et de la commande PWM. Nous définissons une variable statique "commande accumulée" qui garde trace de notre commande lors de l'impulsion précédente, la définition "static" était pour s'assurer que la valeur soit préservée pendant toute la durée de l'exécution.

Nous veillons à sauvegarder la valeur de l'erreur actuelle dans erreur-old. Ensuite, nous implémentons l'une des deux mesures d'*anti-windup* que nous avons utilisées ; celle-ci affecte la commande accumulée, avant division. Nous définissons une variable "rapport" qui ajoute notre commande divisée par 100 (nous préfixons avec *int* pour s'assurer que le résultat de la fraction est entier) à la valeur de base du rapport cyclique (50%). Nous utilisons cette valeur par défaut pour nous assurer que le système a assez d'espace pour augmenter ou diminuer le rapport cyclique selon la perturbation. Nous passons cette valeur par notre second *anti-windup*, cette fois via la fonction "*sature commande*" ; nous faisons cela juste au cas où le fait d'ajouter 50% de rapport cyclique nous ferait sortir de nos limites. Nous injectons cette valeur finale dans notre fonction de réglage du rapport cyclique pour mettre à jour la PWM, et affichons *l'erreur, la commande accumulée et la fréquence*.

Note : la raison pour laquelle nous avons divisé la commande est que nous avons multiplié k_p et k_i , et la raison pour laquelle le facteur était 100 et non 1000 est que nous avons observé une régulation plus rapide avec une commande plus large. Nous avons testé avec une division par 10 mais la commande est alors devenue trop instable.

7.4 | Test :

Pour simuler la perturbation, dans notre cas une température élevée, nous avons utilisé 4 fourchettes, chauffées sur une plaque chauffante, et nous avons alterné entre elles rapidement, une dizaine de secondes à la fois ; nous avons maintenu la chaleur pendant toute la durée du test. Nous avons obtenu le graphique de fréquence suivant :

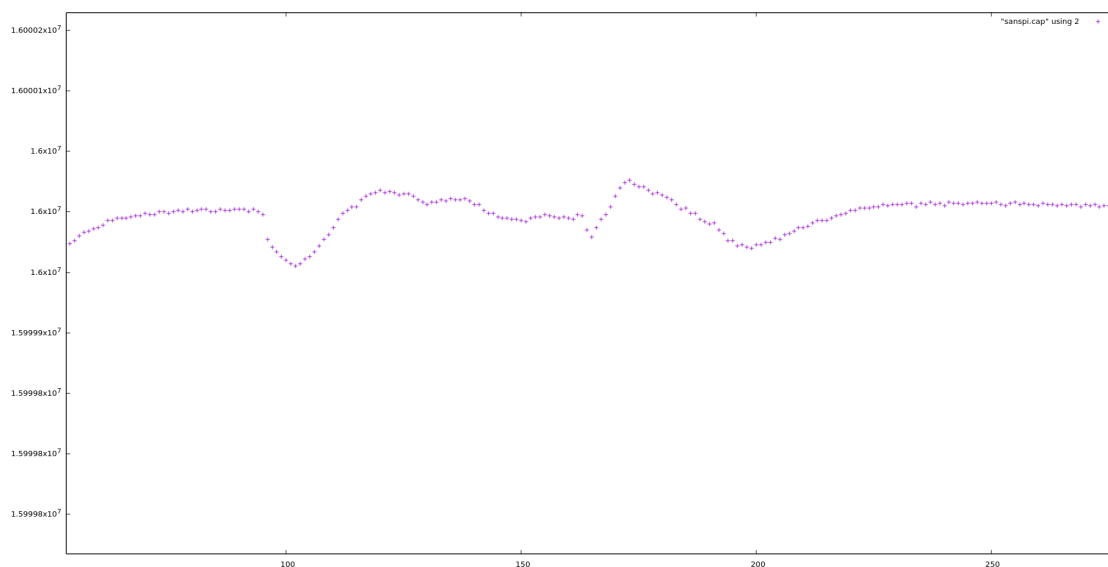


FIGURE 7.3 – Graphique de la variation de fréquence en fonction du temps avec une régulation PI, ou on fait chauffer notre quartz

Nous voyons la chute initiale que nous observons toujours, suivie d'une petite augmentation de fréquence et d'un rétablissement rapide à la consigne. Les variations subséquentes correspondent au changement des fourchettes et à une petite augmentation de température.

Pendant tout le temps où nous avons maintenu une source de chaleur, notre système a été capable de s'adapter et de maintenir fermement notre consigne. Nous considérons cela comme un succès et une régulation efficace.

8 | Conclusion

Ce projet a représenté une opportunité de mettre en pratique concrètement notre compréhension et les connaissances cultivées à travers plusieurs modules durant une semaine de travail, ayant préparé des aspects du projet tels que le schéma, le PCB et le choix des composants ; en les modifiant tout au long du parcours à mesure que nous acquérions une compréhension plus profonde de notre travail.

Nous avons réussi à déterminer avec précision la valeur des composants grâce à la simulation SPICE, soudé des composants délicats avec peu ou pas d'erreur, établi la connexion avec notre carte en nous adaptant aux nouvelles commandes DFU, produit avec précision un signal PWM selon nos besoins, et défini un temps d'échantillonnage via le connecteur coaxial et le générateur, nous amenant à découvrir de nombreux nouveaux registres et leurs fonctions à travers ces deux dernières parties ; le tout en appliquant une régulation qui nous permet d'obtenir un oscillateur robuste, ce qui était notre objectif.

Nous tenons à remercier M. J.-M Friedt et M. E. Carry pour leurs conseils continus, et pour nous avoir permis d'emprunter le générateur afin de terminer les derniers tests sur notre temps personnel.

Table des figures

3.1	Schéma électrique du projet	6
3.2	Liste des empreintes	7
3.3	PCB du schéma électrique	8
3.4	Vues du boîtier modélisé sous FreeCAD	11
3.5	Modélisation 3D du boîtier	11
4.1	Circuit équivalent de la sortie PB5	12
4.2	Résultat de simulation	13
4.3	Schéma spice du résonnateur	14
4.4	Résultat de simulation	15
5.1	Mesure de la tension au niveau du microcontrôleur	16
5.2	Vérification de la carte au microscope	16
6.1	Résultat de la Fast PWM	19
6.2	Signal 1PPS	20
6.3	Résultat de la mesure d'inputCapture	21
7.1	Graphique de l'effet sur la fréquence d'un sweep du rapport cyclique, pour trouver le point de fonctionnement 15 999 958 Hz	22
7.2	Moyenne des 29 mesures effectuées de la réponse indicielle	23
7.3	Graphique de la variation de fréquence en fonction du temps avec une régulation PI, ou on fait chauffer notre quartz	25

9 | Références

- [1] Pierre Curie and Jacques Curie. Développement par compression de l'électricité polaire dans les cristaux hémiedres à faces inclinées. *Comptes Rendus de l'Académie des Sciences*, 1880.
- [2] Paul Langevin. La théorie des oscillations ultrasonores. *Journal de Physique*, 1917.
- [3] Gabriel Lippmann. Principe de la conservation de l'électricité. *Comptes Rendus de l'Académie des Sciences*, 1881.
- [4] Microchip Technology. Atmega16u4/32u4 datasheet. https://www.microchip.com/content/dam/mchp/documents/OTH/ProductDocuments/DataSheets/Atmel-7766-8-bit-AVR-ATmega16U4-32U4_Datasheet.pdf, 2016.
- [5] National Institute of Standards and Technology. Gps time transfer and synchronization principles. <https://www.nist.gov>.
- [6] Pierre Tinguy. Étude et développement d'un oscillateur à quartz intégré. <https://theses.hal.science/tel-00675277v1/document>, 2012.
- [7] Wikipédia. Quartz. <https://fr.wikipedia.org/wiki/Quartz>.